



Department of Software Engineering

Faculty of Computing

Sri Lanka Institute of Information Technology

SE3100 – Architecture Based Development
Year 3, Semester 1 - 2026

Final Report - SE_40

Case Study 01: Sri Lanka Metrobus Service Platform

Team Members

Senarathna Y.M.C.S , IT24013885

Dias M B N S , IT24102244

Ahamed M.N.K , IT24103829

Thilakarathna H K R K , IT24103109

Table of Contents

1. Introduction	8. Member 2: Vehicle Tracking Service
1.1 Case study overview	8.1 C4 Level 3 component diagram
1.2 Scope and assumptions	8.2 Component responsibilities and interactions
2. Requirements analysis	8.3 Design justification
2.1 Functional requirements	8.4 Design completeness
2.2 Quality attributes	8.5 Consistency with Levels 1 and 2
2.3 Constraints	8.6 AI issue 1: synchronous calls in both directions between tracking and the core
3. Quality attribute prioritization and trade-offs	8.7 AI issue 2: no path for delay information to leave the tracking service
3.1 Prioritized quality attributes	9. Member 3: Notification Service
3.2 Trade-offs and tensions	9.1 C4 Level 3 component diagram
3.3 Impact on architectural decisions	9.2 Component responsibilities and interactions
4. Architectural style selection	9.3 Design justification
4.1 Alternatives considered	9.4 Design completeness
4.2 Selected style and justification	9.5 Consistency with Levels 1 and 2
4.3 Decomposition and partitioning	9.6 AI issue 1: no component handled priority
4.4 Communication approach	9.7 AI issue 2: retries with no limit and no expiry
4.5 Data ownership	10. Member 4: Validator Device Software
4.6 Deployment approach	10.1 C4 Level 3 component diagram
4.7 Architecture Decision Records	10.2 Component responsibilities and interactions
5. C4 Level 1: System Context	10.3 Design justification
5.1 Diagram	10.4 Design completeness
5.2 Actors and external systems	10.5 Consistency with Levels 1 and 2
5.3 Relationships	10.6 AI issue 1: offline validation placed in a server component
6. C4 Level 2: Container diagram	10.7 AI issue 2: permission checks placed in the API Gateway
6.1 Diagram	11. Conclusion
6.2 Container responsibilities and technologies	References
6.3 Communication and data stores	Appendix A: AI prompts used
6.4 Traceability to requirements and decisions	A.1 Group review session
7. Member 1: Metrobus Core Platform	A.2 Prompts used by each member
7.1 C4 Level 3 component diagram	
7.2 Component responsibilities and interactions	
7.3 Design justification	
7.4 Design completeness	
7.5 Consistency with Levels 1 and 2	
7.6 AI issue 1: one large Fare and Ticketing component	
7.7 AI issue 2: one shared container for all external integrations	

1. Introduction

1.1 Case study overview

Sri Lanka Metrobus is a proposed metropolitan bus service. Its first phase covers Greater Colombo and links suburbs such as Kottawa, Malabe, Kaduwela, Kadawatha, Ja-Ela, Moratuwa and Panadura with Colombo. Later phases add networks around Kandy, Galle and other cities.

Metrobus needs one digital platform for passenger information, ticketing, fleet operations, route management, journey tracking and service disruptions. The platform serves passengers, several kinds of staff, drivers and partner operators. It also depends on many outside systems that Metrobus does not control.

This report proposes an architecture for that platform. We chose a modular monolith for the core business, with two separate services for vehicle tracking and notifications, and validator devices that work offline. Sections 2 to 4 explain why. Sections 5 to 10 document the design with the C4 model.

1.2 Scope and assumptions

The design covers the first release for Greater Colombo. Future capabilities listed in the case study are not built now, but the design leaves room for them. The case study leaves some details open. We made the assumptions below and marked them so they can be challenged.

ID	Assumption	Why we need it
A1	Passengers tap on when boarding and tap off when leaving.	Fares depend on distance travelled, so the system must know both ends of the trip.
A2	Travel-card balances are kept on the platform, not only on the card.	The case study says travel-card accounts are managed by the new platform.
A3	Validators have local storage and an unreliable mobile data link.	Validation must continue during temporary loss of connectivity.
A4	Each bus sends a GPS position every few seconds. With several hundred buses this is roughly a hundred	The case study says updates are frequent but gives no number.

	updates per second at peak.	
A5	The platform runs on a managed cloud platform with a managed database and message broker.	The technical operations team is small.
A6	Cash is taken by staff or terminals and recorded as a normal ticket sale.	Cash is only required during the initial deployment.

2. Requirements analysis

Every item in this section comes from the case study. The last column of each table names the case study section it comes from.

2.1 Functional requirements

The functional requirements are grouped by area of the case study. Each has an ID that later sections use for traceability.

ID	Requirement	Case study section
Passenger Information		
FR1	Find suitable routes and view nearby stops and interchange points.	Passenger Services
FR2	View scheduled and estimated arrival times.	Passenger Services
FR3	Plan a journey that uses one or more Metrobus routes.	Passenger Services
FR4	View the fare for a journey.	Passenger Services
FR5	View travel history and payment history.	Passenger Services
FR6	Use these services from the mobile app, the website and self-service terminals.	Passenger Services
Ticketing and fares		
FR7	Pay by travel card, mobile app, QR ticket, bank card or digital payment, and by cash in the initial	Ticketing and fares

	deployment.	
FR8	Calculate fares from route, distance travelled, service category and concession eligibility	Ticketing and fares
FR9	Apply transfer rules so that a valid transfer is not charged as two separate journeys.	Ticketing and fares
FR10	Validate tickets while the network is down and synchronize the stored transactions later.	Ticketing and fares
FR11	Manage passenger accounts and travel-card accounts, for registered and occasional passengers.	Passenger Services, Administration and Security
FR12	Send relevant financial transactions to the existing accounting system.	External Systems and Constraints
Network and service planning		
FR13	Manage routes, stops, interchanges and service patterns such as all-stop, express and limited-stop.	Routes and Service Planning
FR14	Manage timetables, frequencies, fares and planned changes, and add extra services quickly.	Routes and Service Planning
Fleet and crew		
FR15	Keep vehicle information: identity, capacity, route suitability, status, assigned service and availability.	Fleet and Driver Operations
FR16	Assign vehicles and crews to duties, and replace an unavailable vehicle without cancelling the service.	Fleet and Driver Operations
FR17	Support directly operated and partner-operated vehicles with the same passenger information and ticketing	Fleet and Driver Operations
FR18	Let drivers see their route, duty, stop and service information on approved mobile or onboard devices.	Fleet and Driver Operations
Live tracking		
FR19	Receive vehicle locations from several GPS providers with different APIs, formats and update rates.	Live Vehicle Tracking

FR20	Keep the latest known position of every vehicle in operation.	Live Vehicle Tracking
FR21	Show passengers approximate bus locations, estimated arrival times, service status and significant delays.	Live Vehicle Tracking
Demand and capacity		
FR22	Show operations staff passenger demand, vehicle occupancy and service usage, and point out heavily loaded services.	Passenger Demand and Capacity
Disruptions		
FR23	Record disruptions that affect a journey, route, stop or a wider part of the network.	Service Disruptions
FR24	Apply a response: delay, replace a vehicle, skip a stop, divert, end early, add a replacement service or suspend a route.	Service Disruptions
FR25	Show the change in passenger information and operational systems as quickly as possible.	Service Disruptions
Notifications		
FR26	Notify passengers by push, SMS and email about delays, diversions, closures, cancellations and card or payment events.	Passenger Notifications
FR27	Send time-sensitive disruption notifications before informational or promotional messages.	Passenger Notifications
Administration and security		
FR28	Give each of the eight staff categories access only to the functions and data they need.	Administration and Security
FR29	Sign in Metrobus employees through the corporate identity provider, and support a different arrangement for partner staff.	Administration and Security

FR30	Record who performed important actions on fares, travel cards, service changes, refunds, route configuration and settings.	Administration and Security
------	--	-----------------------------

2.2 Quality attributes

The quality attributes describe how well the platform must work, not what it does. Each one is linked to the case study text that supports it.

ID	Quality attribute	What it means for Metrobus	Case study evidence
QA1	Availability	Passenger information, ticket validation and core fare processing stay up during operating hours.	Expected Scale: these functions "should remain available throughout normal operating hours".
QA2	Fault isolation	A failure in one part does not spread to unrelated parts.	Tracking failure must not stop ticketing. A notification provider failure must not block the operation. A disruption must not stop unrelated services.
QA3	Operational simplicity	A small team can deploy, monitor and fix the platform.	External Systems and Constraints: small technical operations team, no unnecessary infrastructure and deployment complexity.
QA4	Performance	Validation at the bus door is fast in the peak. Arrival times are fresh. Journey search responds quickly.	Expected Scale: large numbers of validations in short peaks, tens of thousands of searches a day.
QA5	Security	Passenger and payment data are protected. Staff see only what their role allows.	Administration and Security.
QA6	Fare data integrity	Each trip is charged once and correctly, including transfers and offline taps.	Ticketing and Fares: transfers must not be charged incorrectly, offline transactions must be

			synchronized.
QA7	Scalability	The platform handles the gap between peak and offpeak load, and later growth to other cities.	Expected Scale, Future Expansion
QA8	Interoperability	The platform works with outside systems that use different APIs and formats.	Live Vehicle Tracking, External Systems and Constraints.
QA9	Modifiability	New GPS providers, partners, fare products and cities can be added without redesign.	Live Vehicle Tracking, Future Expansion.
QA10	Auditability	Important actions can be traced to the responsible user.	Administration and Security.

2.3 Constraints

2.3.1 Technical constraints

Technical constraints limit the technology choices open to the design.

ID	Constraint	Case study section
TC1	The platform must integrate with outside systems that the team does not control and that may be slow or unavailable.	External Systems and Constraints
TC2	GPS providers use different APIs, formats and update rates, and Metrobus cannot standardize them	Live Vehicle Tracking
TC3	Employees must sign in through the existing corporate identity provider. Partners may use a different arrangement.	Administration and Security
TC4	Validators work on vehicles where network connectivity is not reliable.	Ticketing and Fares

2.3.2 Business constraints

Business constraints come from how Metrobus operates and what it must keep in place.

ID	Constraint	Case study section
BC1	The existing accounting system stays in operation and must receive the financial transactions.	External Systems and Constraints
BC2	Cash must be accepted during the initial deployment.	Ticketing and Fares
BC3	Passengers who do not use the mobile app must still be able to travel with a physical or contactless card.existing corporate identity provider. Partners may use a different arrangement.	Passenger Services
BC4	Some services are run by partner companies with their own vehicles and systems.	Fleet and Driver Operations
BC5	The first release covers Greater Colombo. Future capabilities are not required now but must not be made difficult.	Future Expansion

2.3.3 Operational constraints

Operational constraints describe the conditions the platform must run under.

ID	Constraint	Case study section
OC1	The technical operations team is small. The architecture must avoid unnecessary infrastructure and deployment complexity.	External Systems and Constraints
OC2	Ticketing and active service operations must stay available in operating hours. Reporting and non-critical administration may be interrupted	Expected Scale

OC3	Load is very different between peak and offpeak periods.	Expected Scale
-----	--	----------------

3. Quality attribute prioritization and trade-offs

3.1 Prioritized quality attributes

The ranking shows how strongly each quality attribute shapes the structure of the system. A low rank does not mean the attribute is ignored. It means it can be met without changing the overall structure.

Rank	Quality attribute	Why it has this rank
1	Availability (QA1)	The case study names the functions that must stay up: passenger information, validation and fare processing. A bus that cannot validate tickets cannot collect revenue.
2	Fault isolation (QA2)	The case study states three separate rules about failures that must not spread. No other quality attribute is repeated this often.
3	Operational simplicity (QA3)	Management states it directly. It limits how we are allowed to achieve ranks 1 and 2.
4	Performance (QA4)	Peak boarding and live arrival times are the moments passengers notice. Poor performance here looks like unavailability.
5	Security (QA5)	Payment and personal data are involved, and there are eight staff roles and three identity sources.
6	Fare data integrity (QA6)	Charging must be correct, but the case study accepts a delay: offline transactions are synchronized later.
7	Scalability (QA7)	The first release is several hundred buses in one city. The real need is handling the peak, not unlimited growth.
8	Interoperability (QA8)	There are many outside systems, but each can be handled with an adapter in one place.
9	Modifiability (QA9)	The case study says future capabilities are not required in the first release. The design must only avoid blocking

		them.
10	Auditability (QA10)	It is mandatory but it is met by one component and does not shape the structure.

The top three decide the architecture. Availability and fault isolation push towards separating parts. Operational simplicity pushes towards fewer parts.

3.2 Trade-offs and tensions

T1. Availability against fare data integrity.

If a validator must ask the server before accepting a card, charging is always correct, but validation stops when the network drops. The case study requires validation to continue offline. We put availability first. Validators decide locally and the server corrects the accounts when transactions arrive. The cost is a small revenue risk: a card with no balance may be accepted while the device is offline.

T2. Fault isolation against operational simplicity.

Every part that runs separately can fail separately, which is good for isolation. Every separate part also needs its own deployment, monitoring and on-call knowledge. A small team cannot run many of them. We separate only the two parts where the case study states an isolation rule: vehicle tracking and notifications.

T3. Scalability against operational simplicity.

Microservices would let every function scale alone. The case study load does not need that. Only tracking has a load pattern different from the rest. The core is scaled as one unit by running more copies of it in the peak.

T4. Performance against fault isolation.

Passengers want live arrival times, which come from the tracking service. If journey planning waits on tracking for every request, a tracking failure slows or breaks journey planning. We accept less accurate data during a failure. Journey planning falls back to timetable times and marks them as scheduled.

T5. Security against availability at the edge.

A lost or blocked card should be rejected at once. An offline validator cannot check the server. We accept a short delay. Validators keep a local list of blocked cards and refresh it every time they connect.

T6. Modifiability against operational simplicity.

Running a separate system for each future city would make cities fully independent. It would also multiply the work of the operations team before a second city exists. We record the network as data inside one system and leave separate deployment as a later option.

3.3 Impact on architectural decisions

Tradeoff	Side we favour	Resulting decision	Recorded in
T1	Availability	Validators work offline first and synchronize later.	ADR-004
T2	Balanced	Modular monolith for the core. Tracking and notifications are separate services.	ADR-001, ADR-002, ADR-003
T3	Operational simplicity	The core scales as a whole. Reporting reads from a database replica.	ADR-001, ADR-009
T4	Fault isolation	Journey planning has a timetable fallback. Tracking sends delay events instead of being polled.	ADR-002
T5	Availability	Blocked-card list is held on the device and refreshed on each connection.	ADR-004
T6	Operational simplicity	Each metropolitan network is a data attribute, not a separate deployment.	ADR-008

4. Architectural style selection

4.1 Alternatives considered

Several architectural styles were compared against the needs of the case study before one was selected.

Option	What it is	Fit with the case study	Verdict
Layered monolith	One deployable, split into technical layers.	Simplest to run. But a GPS burst or a slow provider shares one process with ticketing, which breaks QA2. A new fare product would touch every layer	Rejected
Microservices	One small service and database for each business function.	Best isolation and independent scaling. But many deployments, distributed fare transactions and heavy monitoring. This breaks OC1.	Rejected
Fully event-driven	All parts talk only through events.	Good decoupling. But buying a ticket or editing a timetable needs an immediate answer, and event-only flows are hard for a small team to trace.	Used in part only
Modular monolith with selected services	One core deployable split into domain modules, plus a few services where the case study demands separation	Keeps operations simple and still isolates the parts named in the case study.	Selected

4.2 Selected style and justification

We selected a domain-partitioned modular monolith for the core business, with two separately deployed services (Vehicle Tracking and Notification), event-driven messaging between them, and offline-first validator devices on the vehicles.

The core is a monolith because most Metrobus functions share the same data and change together. Fares need routes and stops. Disruptions change services and vehicles. Keeping

them in one deployable means simple transactions and one thing to deploy, which fits the small team (OC1, QA3).

Vehicle tracking is separate because the case study says its failure must not stop ticketing or unrelated routes (QA2). It also deals with provider differences that should not leak into the core (TC2). With our estimate in A4, the volume alone would not force a separate service. Isolation and provider variety are the real reasons.

Notifications are separate because a provider failure must not block the operation that caused the message (QA2, TC1). Messaging between the core and this service is asynchronous for the same reason.

Validators work offline first because validation must continue without the network (FR10, TC4). This is what protects availability (QA1) even if the whole back end is unreachable.

4.3 Decomposition and partitioning

The core is partitioned by domain, not by technical layer. Changes in the case study arrive by domain: a new fare product, a new partner operator, a new disruption response. With domain partitioning each of these changes stays inside one module.

Module	Responsibility	Changes when
Passenger Accounts	Passenger profiles, registration, history views, notification preferences	Account features change
Fares and Ticketing	Fare rules, travel-card accounts, tickets, payments, validation synchronization	Fare products or payment methods change
Network Planning	Routes, stops, interchanges, service patterns, timetables	The network or timetable changes
Journey Planning	Route search and journey plans	Search features change
Fleet and Duty	Vehicles, crews, duties, replacements, partner vehicles	Fleet or partner arrangements change
Disruption	Disruptions and the responses	Operating procedures

Management	applied	change
Demand and Reporting	Demand, occupancy, usage reports	Reporting needs change
Access and Administration	Roles, permissions, staff mapping, settings	Staff structure changes
Audit	Record of important actions	Rarely

4.4 Communication approach

The table lists how each pair of parts communicates and why that method was chosen.

Between	Style	Why
Modules inside the core	Direct in-process calls	Fast, and one database transaction can cover a fare operation.
Client applications and the back end	HTTPS requests through the API Gateway	Users need an immediate answer.
Core to Notification Service	Asynchronous events through the Message Broker	A provider failure must not block the core.
Tracking Service to core	Asynchronous events through the Message Broker	The core learns about delays without depending on tracking being up.
Core to Tracking Service	Asynchronous events through the Message Broker	Tracking keeps its own copy of timetable and vehicle assignment data.
Journey Planning to Tracking Service	HTTPS request with a short timeout and a timetable fallback	Live times are wanted, but must not be required.
Validators and the back end	Batch synchronization when connected	The network on vehicles is unreliable.
Core to outside systems	Calls through adapters with timeouts. Accounting uses a stored queue with retries.	Outside systems may be slow or down.

4.5 Data ownership

Each type of data has exactly one owner. Other parts read it through the owner's interface or receive a copy through events.

Data	Owner	Stored in	How others get it
Passengers, cards, fares, tickets, routes, timetables, vehicles, duties, disruptions, roles, audit records	The matching core module	Core Database, one schema per module	Core Database, one schema per module
Vehicle positions, arrival estimates, provider status	Vehicle Tracking Service	Tracking Data Store	Tracking query interface and delay events
Notification subscriptions copy, delivery log	Notification Service	Notification Data Store	Not shared. The source of subscriptions is PassengerAccounts.
Unsent taps, plus copies of fare tables and the blocked-card list	Validator Device Software	Local device store	Synchronized with Fares and Ticketing

Each piece of data has one owner. Copies exist in three places: timetable data in the tracking service, subscriptions in the notification service, and fare tables and the blocked-card list on validators. The first two are refreshed by events and the third by synchronization, so each part keeps working if the core is briefly unavailable.

4.6 Deployment approach

The team operates three back-end deployables: the Core Platform, the Vehicle Tracking Service and the Notification Service. The API Gateway, databases and Message Broker are managed services (A5). The Core Platform runs as at least two identical copies behind the gateway, so one copy can fail or be updated without stopping ticketing. More copies are added in the peak and removed off-peak (OC3). The Core Database has a standby copy for failover and a read replica for reporting. This is a distributed deployment, but a small one.

Each separately deployed part is justified by a specific case study statement, which Section 4.7 records.

4.7 Architecture Decision Records

Nine decisions are recorded in the MADR format. The summary table lists them. Each record then gives the context, the decision drivers, the options considered, the outcome and the consequences.

ADR	Decision	Main reason
ADR-001	Domain-partitioned modular monolith for the core	The operations team is small
ADR-002	Separate Vehicle Tracking Service with its own data store	Tracking failure must not stop ticketing
ADR-003	Asynchronous notifications through a separate service	A provider failure must not block an operation
ADR-004	Offline-first validation on the device	Validation must continue without the network
ADR-005	One core database with one schema per module	Clear data ownership with simple operations
ADR-006	Each external adapter inside the container that uses it	No shared point of failure
ADR-007	Three identity sources, with role checks in the core	Existing identity provider and eight staff roles
ADR-008	Each metropolitan network modelled as data	Expansion without extra deployments
ADR-009	Critical functions protected inside the monolith	Ticketing matters more than reporting

ADR-001: Use a domain-partitioned modular monolith for the core

Field	Detail
Status	Accepted
Context and problem	Most Metrobus functions share data and change together. The team that runs the platform is small. How should the core business functions be structured and deployed?
Decision drivers	Small operations team (OC1). Operational simplicity (QA3). Fare data integrity (QA6). Modifiability (QA9).
Considered options	(1) Layered monolith. (2) Microservices. (3) Modular monolith partitioned by domain.
Decision outcome	Option 3. It gives one deployable and simple fare transaction, and it keeps clear module boundaries.
Good consequences	There is one thing to deploy and monitor. A fare operation is one database transaction. A module can be extracted later because it already owns its data.
Bad consequences	A serious fault in one module can affect the whole core, which ADR-009 reduces. Module boundaries depend on team discipline and code checks.

ADR-002: Run vehicle tracking as a separate service with its own data store

Field	Detail
Status	Accepted
Context and problem	Hundreds of vehicles send positions through providers with different APIs and formats. Tracking failure must not prevent ticket validation, fare processing or unrelated routes. Where should tracking run?
Decision drivers	Fault isolation (QA2). Performance (QA4). Different GPS providers (TC2). Tracking requirements (FR19 to FR21).
Considered options	(1) A tracking module inside the core. (2) A separate service that shares the Core Database. (3) A separate service with its own data store.

Decision outcome	Option 3. Provider adapters live inside the service. It publishes delay events and keeps its own copy of timetable data.
Good consequences	A tracking or provider failure cannot reach ticketing. Constant position writes do not compete with fare transactions. database transaction. A module can be extracted later because it already owns its data.
Bad consequences	There is one more deployable to operate. Timetable data is copied and can be briefly out of date.

ADR-003: Deliver notifications asynchronously through a separate service

Field	Detail
Status	Accepted
Context and problem	SMS, email and push providers may be slow or unavailable. Their failure must not prevent the operational change or ticketing transaction. Urgent messages must go before promotional ones. How should notifications be sent?
Decision drivers	Fault isolation (QA2). Unreliable outside systems (TC1). Notification requirements (FR26, FR27).
Considered options	(1) The core calls providers directly during the operation. (2) The core sends in a background job in the same process. (3) A separate Notification Service fed by a Message Broker.
Decision outcome	Option 3, with a separate queue for each priority.
Good consequences	The core finishes its work without waiting for any provider. Urgent messages are not stuck behind bulk messages.
Bad consequences	A broker must be operated, which we reduce by using a managed one. Delivery is delayed, and it can fail after the operation has succeeded.

ADR-004: Validate tickets offline first on the device

Field	Detail
Status	Accepted
Context and problem	Validators lose connectivity on the road. Validation must continue and transactions must be synchronized later. Where is the accept or reject decision made?
Decision drivers	Availability (QA1). Offline validation (FR10). Unreliable network on vehicles (TC4). Fare data integrity (QA6). Security (QA5).
Considered options	(1) Always ask the server. (2) Ask the server and fall back to local rules when offline. (3) Always decide locally and synchronize in the background.
Decision outcome	Option 3. The device holds fare tables, a blockedcard list and the key for checking QR tickets. It stores each tap with a unique ID and uploads when connected. The server calculates the final fare and applies transfer rules.
Good consequences	Boarding speed is the same with or without a network. Ticketing survives a back-end outage.
Bad consequences	A card with no balance may be accepted until the next synchronization. Devices need secure storage and a way to update their data

ADR-005: One core database with one schema per module

Field	Detail
Status	Accepted
Context and problem	Modules in the monolith need storage. Shared tables would hide coupling. Separate databases would add operational work. How should core data be organized?
Decision drivers	Operational simplicity (QA3). Fare data integrity (QA6). Modifiability (QA9).
Considered	(1) One shared schema. (2) One database with one schema per module.

options	(3) One database per module.
Decision outcome	Option 2. A module reads and writes only its own schema.
Good consequences	There is one database to back up and fail over. Transactions across modules stay possible where the business needs them. A module can move to its own database later.
Bad consequences	The database is shared infrastructure, so ADR-009 must protect it.

ADR-006: Keep each external adapter inside the container that uses it

Field	Detail
Status	Accepted
Context and problem	The platform integrates with seven kinds of outside system, and they may be slow or down. Where does the integration code live?
Decision drivers	Fault isolation (QA2). Interoperability (QA8). Unreliable outside systems (TC1). Existing accounting system (BC1).
Considered options	(1) One shared integration container for all outside systems. (2) An adapter inside each container that needs the outside system.
Decision outcome	Option 2. GPS adapters are in the Tracking Service. Channel adapters are in the Notification Service. Payment, accounting and partner adapters are in the core. Accounting transfers go into a stored queue and are sent with retries.
Good consequences	A failing GPS adapter cannot affect payments. There is no extra container to operate. A slow accounting system never blocks a fare transaction.
Bad consequences	Common concerns such as timeouts must be applied consistently in three places.

ADR-007: Use three identity sources and authorize by role in the core

Field	Detail
Status	Accepted
Context and problem	Employees use the corporate identity provider. Partner staff may sign in differently. Passenger accounts are managed by the platform. Eight staff categories need different access. How are users identified and authorized?
Decision drivers	Security (QA5). Existing identity provider (TC3). Access requirements (FR28, FR29). Auditability (QA10).
Considered options	(1) One platform-managed login for everyone. (2) Federated sign-in for staff, with platform-managed passenger accounts.
Decision outcome	Option 2. Employees sign in through the corporate identity provider and partner staff through the partner arrangement. Passengers sign in to platform accounts. The API Gateway checks the token. The core maps each user to roles and enforces permissions.
Good consequences	No staff passwords are stored in the platform. Permission rules sit beside the business logic they protect.
Bad consequences	Staff sign-in depends on an outside identity provider being available.

ADR-008: Model each metropolitan network as data

Field	Detail
Status	Accepted
Context and problem	Metrobus will add separate networks around Kandy, Galle and other cities. The first release covers Greater Colombo only. How should the design prepare for this?
Decision drivers	Future expansion (BC5). Modifiability (QA9). Small operations team (OC1).
Considered options	(1) A separate deployment for each city. (2) One deployment where every route, stop, vehicle and fare belongs to a network.

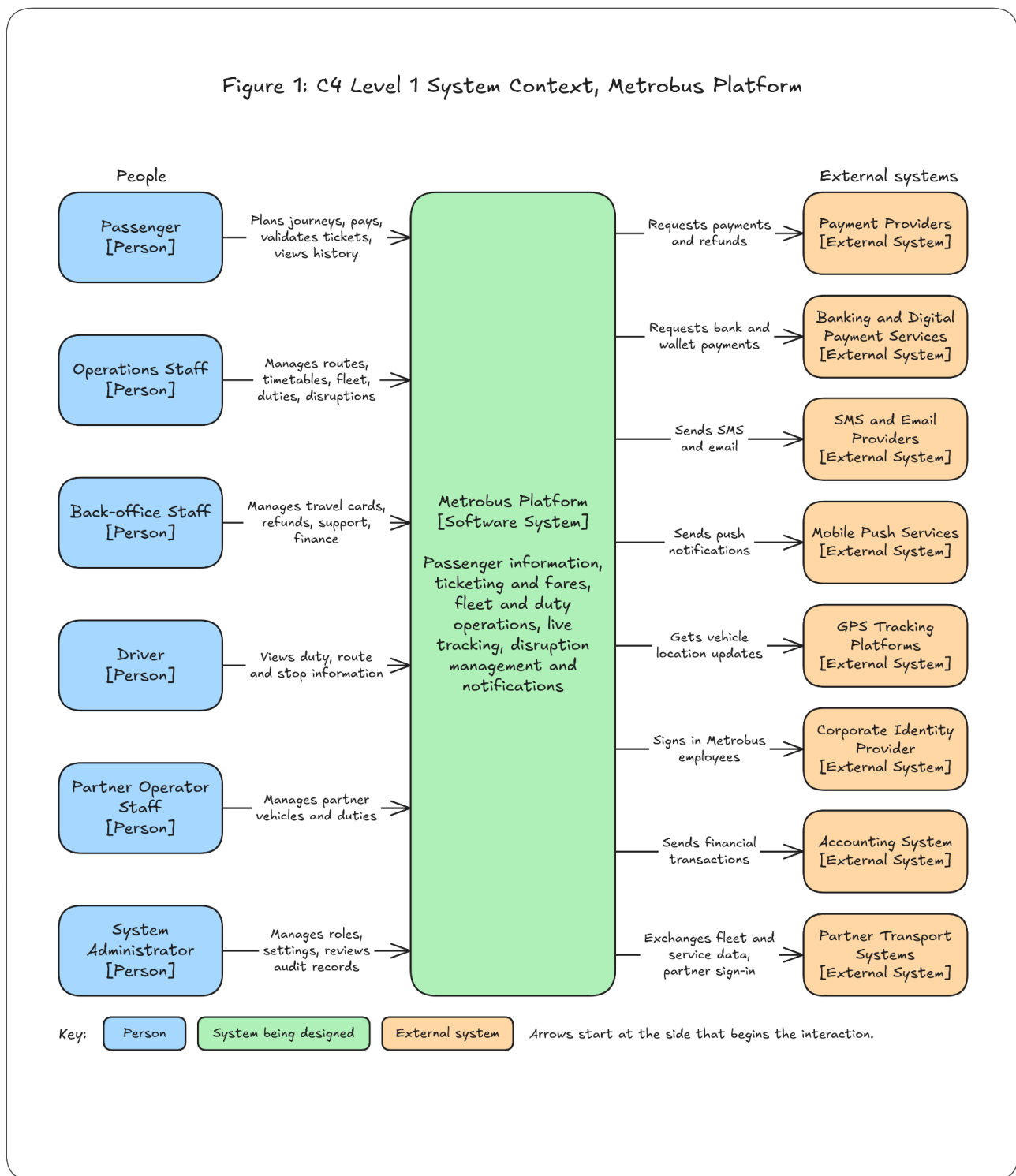
Decision outcome	Option 2. Network-specific data carries a network identifier, and staff permissions can be limited to a network.
Good consequences	A new city is a configuration task, not a new system. Integrated fares across networks stay possible.
Bad consequences	A fault in the shared core affects all cities. Separate deployment remains a later option because modules own their data.

ADR-009: Protect critical functions inside the monolith

Field	Detail
Status	Accepted
Context and problem	Ticketing must stay available, while reporting and non-critical administration may be interrupted. All of them live in one core. How do we stop the less important work from harming the critical work?
Decision drivers	Availability (QA1). Critical functions must stay up (OC2). Peak and off-peak load (OC3).
Considered options	(1) Extract ticketing into its own service. (2) Keep one core and protect the critical paths.
Decision outcome	Option 2. Run at least two copies of the core with no session data stored in them. Keep a standby database. Send reporting queries to a read replica. Run heavy background jobs outside peak hours.
Good consequences	One copy can fail or be updated without stopping ticketing. A heavy report cannot slow fare processing. Validators already cover a short full outage (ADR-004).
Bad consequences	Reports may be a few seconds behind live data.

5. C4 Level 1: System Context

5.1 Diagram



Full-resolution diagram:

<https://github.com/chamals3n4/se3100-metrobus-architecture/blob/main/C4-Diagrams/Level-1-System-Context/SE40-C4-L1-System-Context.png>

5.2 Actors and external systems

The table lists every person and outside system in the context diagram, with its role.

Element	Type	Description
Passenger	Person	Registered or occasional traveller. Uses the app, website, terminals, travel cards and QR tickets
Operations Staff	Person	Operations controllers, route planners and fleet administrators.
Back-office Staff	Person	Ticketing staff, customer-support staff and finance staff.
Driver	Person	Drives a Metrobus or partner vehicle and views duty information.
Partner Operator Staff	Person	Staff of an approved partner company who manage its vehicles and duties.
System Administrator	Person	Manages roles, settings and reviews audit records.
Payment Providers	External system	Process card and online payments and refunds.
Banking and Digital Payment Services	External system	Process bank card and digital wallet payments.
SMS and Email Providers	External system	Deliver SMS and email messages.
Mobile Push Services	External system	Deliver push notifications to passenger phones.
GPS Tracking Platforms	External system	Supplier and partner tracking systems that expose vehicle locations.
Corporate Identity Provider	External system	Existing sign-in system for Metrobus employees.
Accounting System	External system	Existing finance system that receives financial transactions.

Partner Transport Systems	External system	Systems of partner operators, including their staff sign-in arrangement.
---------------------------	-----------------	--

5.3 Relationships

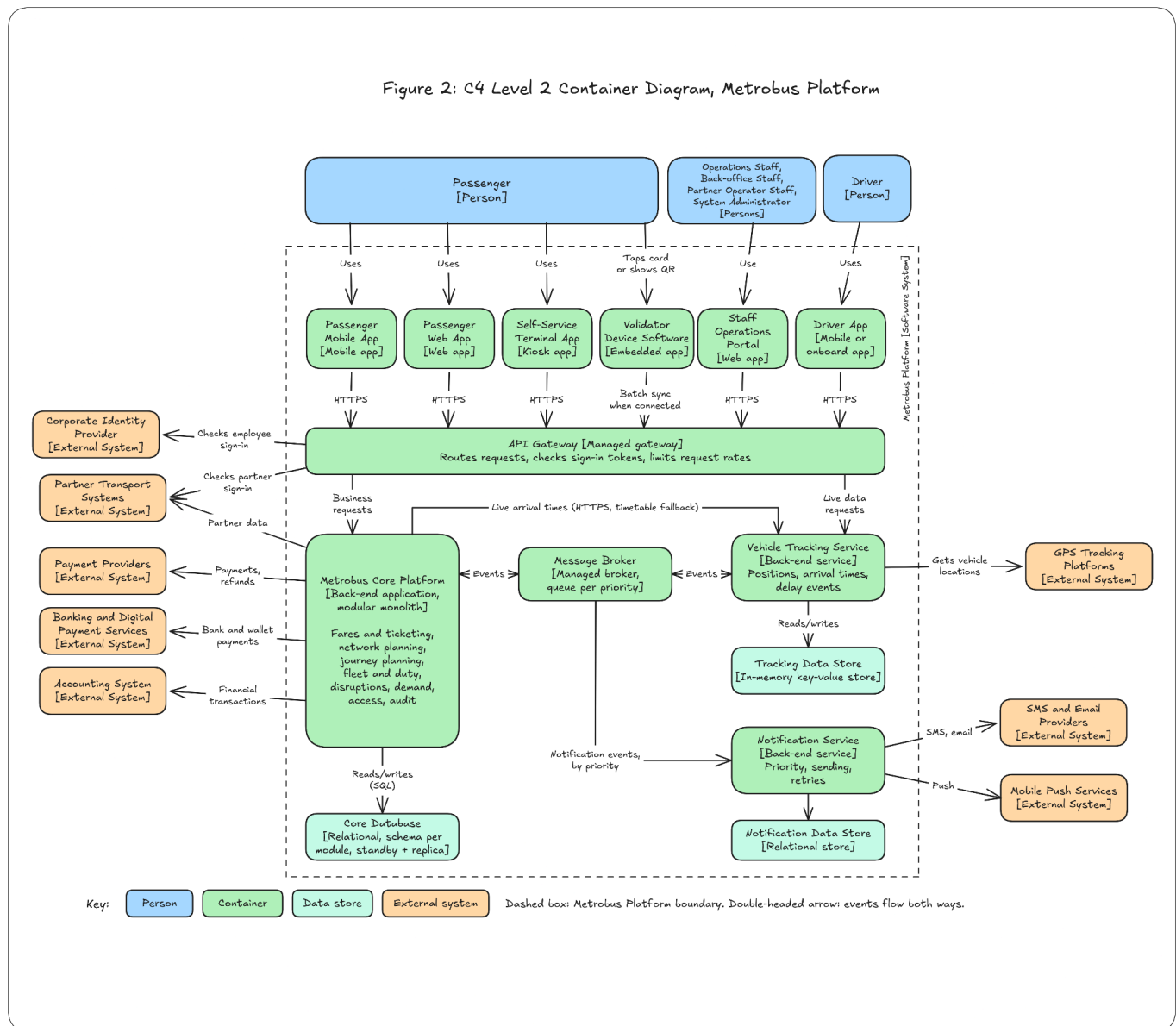
Each arrow in the context diagram is listed below with what passes along it.

From	To	Label on the arrow
Passenger	Metrobus Platform	Plans journeys, views arrival times and fares, pays, validates tickets, views history
Operations Staff	Metrobus Platform	Manages routes, timetables, fleet, duties and disruptions, views demand
Back-office Staff	Metrobus Platform	Manages travel cards, refunds, support requests and financial records
Driver	Metrobus Platform	Views assigned duty, route, stop and service information
Partner Operator Staff	Metrobus Platform	Manages partner vehicles and duties
System Administrator	Metrobus Platform	Manages roles and settings, reviews audit records
Metrobus Platform	Payment Providers	Requests payments and refunds
Metrobus Platform	Banking and Digital Payment Services	Requests bank and wallet payments
Metrobus Platform	SMS and Email Providers	Sends SMS and email notifications
Metrobus Platform	Mobile Push Services	Sends push notifications
Metrobus Platform	GPS Tracking Platforms	Gets vehicle location updates
Metrobus Platform	Corporate Identity Provider	Signs in Metrobus employees

Metrobus Platform	Accounting System	Sends financial transactions
Metrobus Platform	Partner Transport Systems	Exchanges vehicle, duty and service data, signs in partner staff

6. C4 Level 2: Container diagram

6.1 Diagram



Full-resolution diagram:

<https://github.com/chamals3n4/se3100-metrobus-architecture/blob/main/C4-Diagrams/Level-2-Container/SE40-C4-L2-Container-Diagram.png>

6.2 Container responsibilities and technologies

The platform is made up of the containers below. Each one runs separately and has its own technology choice.

Container	Type	Responsibility
Passenger Mobile App	Mobile app	Journey planning, arrival times, fares, QR tickets, card top-up, history and push notifications.
Passenger Web App	Web app in the browser	The same passenger information and account services on the website.
Self-Service Terminal App	Kiosk app at major terminals	Passenger information, card topup and ticket purchase without the mobile app
Staff Operations Portal	Web app	All staff functions, shown according to the user's role. Used by operations, back-office, partner and administrator users.
Driver App	Mobile or onboard app	Shows the driver's duty, route, stops, service information and disruption instructions.
Validator Device Software	Embedded app with a local store, on buses and terminal gates	Accepts or rejects cards and QR tickets without the network. Stores taps and synchronizes them later.
API Gateway	Managed gateway	Accepts or rejects cards and QR tickets without the network. Stores taps and synchronizes them later.
API Gateway	Managed gateway	Single entry point. Routes requests, checks sign-in tokens and limits request rates. Holds no business logic.
Metrobus Core Platform	Back-end application, modular monolith, two or more copies	All core business modules listed in Section 4.3.
Vehicle Tracking Service	Back-end service	Gets GPS data from providers, converts it to one format, keeps latest positions, estimates arrival times and reports delays

Notification Service	Back-end service	Receives events, decides who to notify and how urgently, sends by push, SMS and email, and retries failures.
Message Broker	Managed broker with a separate queue for each priority	Carries events between the core, the tracking service and the notification service
Core Database	Relational database, one schema per module, with a standby and a read replica	Stores all core business data.
Tracking Data Store	In-memory key-value store with short history	Stores latest vehicle positions, arrival estimates and provider status
Notification Data Store	Small relational store	Stores the subscriptions copy and the delivery log.

6.3 Communication and data stores

All client applications reach the back end through the API Gateway over HTTPS. The gateway checks the sign-in token and passes the request to the Core Platform, or to the Vehicle Tracking Service for live positions and arrival times. Validators work differently: they upload stored taps and download updates in batches whenever they have a connection.

The back-end parts talk to each other through the Message Broker. The core publishes disruption, ticketing, timetable and subscription events. The tracking service publishes delay events. The Notification Service only receives events. There is one direct call, from the core to the tracking service for live arrival times, and it falls back to timetable times if tracking does not answer.

Each back-end part has its own data store, and no other part reads it. The core uses the Core Database, the tracking service uses the Tracking Data Store, and the Notification Service uses the Notification Data Store.

Each outside system is reached by the part that needs it. The tracking service gets positions from the GPS platforms. The Notification Service sends through the SMS, email and push providers. The core calls the payment and banking services, exchanges data with partner systems, and sends financial transactions to the accounting system through a stored queue with retries. There is no shared integration container, as decided in ADR006

6.4 Traceability to requirements and decisions

Each container exists because of specific requirements and decisions.

The passenger apps and the terminal deliver the passenger services (FR1 to FR7). The Staff Operations Portal and the DriverApp deliver the staff and driver functions (FR13 to FR18, FR22 to FR24). The Validator Device Software delivers offline validation (FR10) and follows ADR-004.

The Core Platform holds the business rules for fares, the network, the fleet, disruptions, access and audit. It follows ADR-001, ADR-005 and ADR-009. The Vehicle Tracking Service covers live tracking (FR19 to FR21) and follows ADR-002. The Notification Service and the Message Broker cover notifications (FR26, FR27) and follow ADR-003. The API Gateway supports sign-in (FR29) and follows ADR-007.

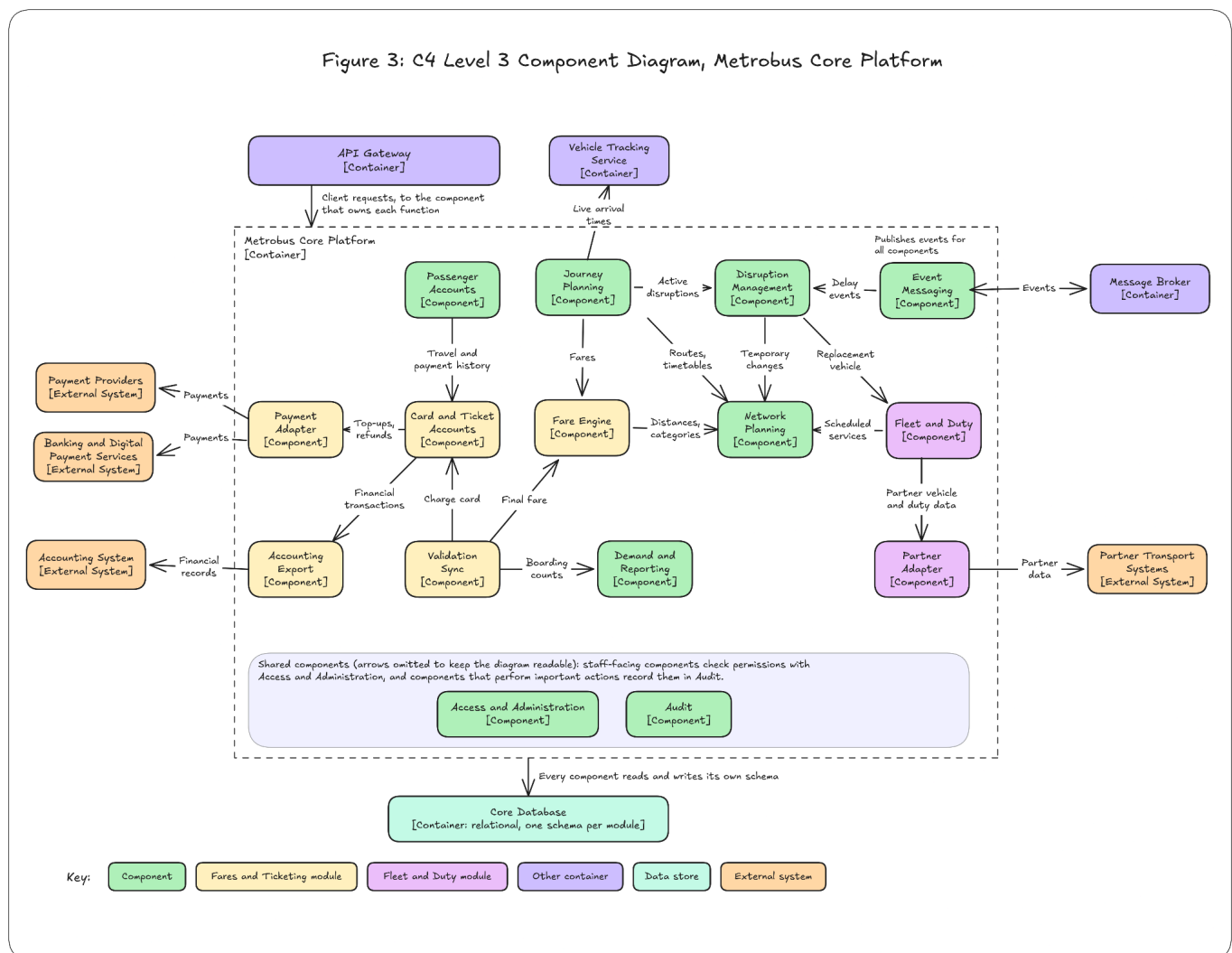
Every person in Figure 1 has an entry point in Figure 2. Passengers use the passenger apps, the terminal and the validators. Staff use the Staff Operations Portal. Drivers use the DriverApp. Every external system in Figure 1 also appears in Figure 2

7. Member 1: Metrobus Core Platform

Responsible member: Senarathna Y.M.C.S (IT24013885)

7.1 C4 Level 3 component diagram

Figure 3 opens the Metrobus Core Platform container from Figure 2. It shows the 15 components inside the core, with the neighbouring containers and outside systems around the boundary. Requests enter only from the API Gateway, each component uses its own schema in the Core Database, and events leave only through Event Messaging. Outside systems are reached only through Payment Adapter, Accounting Export and Partner Adapter.



Full-resolution diagram:

<https://github.com/chamals3n4/se3100-metrobus-architecture/blob/main/C4-Diagrams/Level-3-Components/SE40-C4-L3-Core-Platform.png>

7.2 Component responsibilities and interactions

Components. The core has 15 components.

Component	Responsibility
Passenger Accounts	Passenger profiles, registration, notification preferences, and the travel and payment history view.
Fare Engine	The only place that calculates a fare. Applies route, distance, service category, concession and transfer rules.
Card and Ticket Accounts	Travel-card accounts and balances, QR ticket issue, top-ups, refunds and the blocked-card list.
Validation Sync	Receives taps uploaded by validators, removes duplicates, builds trips and sends fare tables and the blocked-card list to devices.
Payment Adapter	Talks to payment providers and banking services. Applies timeouts.
Accounting Export	Keeps a stored queue of financial transactions and sends them to the accounting system with retries.
Network Planning	Routes, stops, interchanges, service patterns, timetables, frequencies and planned changes.
Journey Planning	Finds routes and builds journey plans with fares, arrival times and active disruptions
Fleet and Duty	Vehicles, crews, duty assignment and vehicle replacement, for Metrobus and partner vehicles.
Partner Adapter	Exchanges vehicle, duty and service data with partner transport systems.
Disruption Management	Records a disruption with its scope and applies the chosen response.
Demand and Reporting	Builds demand, occupancy and usage views from boarding data. Reads from the database replica.
Access and Administration	Maps signed-in users to roles, checks permissions and holds system settings
Audit	Keeps an add-only record of important actions and who performed them.

Event Messaging	Stores outgoing events in the database, publishes them to the Message Broker and passes incoming delay events to Disruption Management.
-----------------	---

Interactions. Client requests arrive from the API Gateway and go to the component that owns the function. Components that serve staff first check the user's permission with Access and Administration.

Journey Planning reads routes and timetables from Network Planning, fares from the Fare Engine and active disruptions from Disruption Management. It asks the Vehicle Tracking Service for live arrival times and uses timetable times if there is no answer.

Validation Sync receives taps from the validators. It asks the Fare Engine for the final fare, tells Card and Ticket Accounts to charge the card, and passes boarding counts to Demand and Reporting. The Fare Engine reads distances and service categories from Network Planning.

Card and Ticket Accounts uses the Payment Adapter for top-ups and refunds, and adds each financial transaction to Accounting Export. PassengerAccounts gets travel and payment history from Card and Ticket Accounts.

Disruption Management applies temporary changes through Network Planning and requests replacement vehicles from Fleet and Duty. Fleet and Duty reads scheduled services from Network Planning and exchanges partner data through the PartnerAdapter.

Components publish events through Event Messaging, which is the only component that talks to the Message Broker. Components that perform important actions record them in Audit. Every component stores its data in its own schema in the Core Database.

Only three components talk to outside systems: the Payment Adapter (payment and banking services), Accounting Export (accounting system) and the PartnerAdapter (partner systems).

7.3 Design justification

Each design choice below is linked to the requirements, quality attributes or decisions that justify it.

Design choice	Justified by
Fares and Ticketing is opened into five components.	Fare rules will change often as passes, concessions and integrated fares arrive. Card accounts and payments are stable. Each part now has one reason to change (QA9, FR8, FR9).
The Fare Engine is the only component that calculates fares.	The same trip must cost the same on every channel, and transfers must not be charged twice (FR9, QA6).
Validation Sync is a separate component that removes duplicate taps.	Devices may upload the same tap twice after a lost connection. Each tap has a unique ID, so it is charged once (FR10, QA6).
Outside systems are reached only through Payment Adapter, Accounting Export and Partner Adapter.	Outside systems may be slow or down. Their problems stay in one component each (TC1, QA2, ADR-006).
Accounting Export uses a stored queue.	The accounting system stays in operation and must receive the transactions, but it must not block a fare (BC1, TC1).
Disruption Management changes the network through Network Planning's interface and stores the scope of each disruption.	A disruption in one area must not affect unrelated services, and modules must not write each other's data (QA2, ADR-005).
Demand and Reporting reads from the database replica.	Reporting may be interrupted but must never slow ticketing (OC2, ADR-009).
Access and Administration is the single place for permission checks.	Eight staff categories must see only what they need (FR28, QA5).
Audit is one add-only component.	Important actions must be traceable to a user (FR30, QA10).
Event Messaging stores an event in the	A business change and its event are saved

database before publishing it.	together, so no event is lost if the broker is down (QA2, FR25).
--------------------------------	--

7.4 Design completeness

Requirement coverage. Every functional requirement owned by the core has a component.

Requirements	Component
FR1, FR3, FR4	Journey Planning, with Network Planning and Fare Engine
FR5, FR11	Passenger Accounts, Card and Ticket Accounts
FR7	Card and Ticket Accounts, Payment Adapter
FR8, FR9	Fare Engine
FR10 (server side)	Validation Sync
FR12	Accounting Export
FR13, FR14	Network Planning
FR15, FR16, FR18	Fleet and Duty
FR17	Fleet and Duty, Partner Adapter
FR22	Demand and Reporting
FR23, FR24, FR25	Disruption Management, Event Messaging
FR28, FR29	Access and Administration
FR30	Audit

Scenario 1: a transfer journey recorded offline.

1. The passenger taps on and off bus A, then taps on bus B. Both validators are offline and store the taps.
2. The validators reconnect and upload the taps to Validation Sync.
3. Validation Sync ignores any tap ID it has already processed and groups the taps into trips.
4. The Fare Engine reads distances from Network Planning and applies the transfer rule. The result is one journey fare.

5. Card and Ticket Accounts charges the card and adds a record to Accounting Export in the same database transaction.
6. Validation Sync passes the boarding counts to Demand and Reporting.

Scenario 2: a vehicle breaks down in service.

1. An operations controller records the breakdown in Disruption Management. The permission is checked and the action is audited.
2. Disruption Management asks Fleet and Duty for an available vehicle with suitable capacity and route suitability.
3. Fleet and Duty assigns the replacement and updates the duty. The Driver App shows the new duty.
4. Event Messaging publishes the disruption and the new assignment.
5. The Tracking Service updates its assignment copy. The Notification Service informs affected passengers.

Failure handling.

Failure	What the core does
Payment provider is slow or down	Payment Adapter times out. Top-ups fail with a clear message. Card taps still work because balances are held in the platform.
Accounting system is down	Records wait in the stored queue and are sent later.
Vehicle Tracking Service is down	Journey Planning shows timetable times marked as scheduled.
Message Broker is down	Events wait in the database and are published when the broker returns.
One copy of the core fails	The gateway sends requests to the other copy.
A heavy report runs in the peak	It runs on the replica and does not touch fare processing.

7.5 Consistency with Levels 1 and 2

Each neighbouring element from the higher-level diagrams is checked against this design below.

Level 2 element	How it appears at Level 3
API Gateway	The only source of client requests into the core.
Core Database	Used by every component, each with its own schema.
Message Broker	Reached only through Event Messaging.
Vehicle Tracking Service	Called only by Journey Planning. Its delay events arrive through Event Messaging.
Payment Providers, Banking and Digital Payment Services	Reached only through Payment Adapter.
Accounting System	Reached only through Accounting Export.
Partner Transport Systems	Reached only through Partner Adapter.

The diagram contains no neighbour that is missing from Figure 2. GPS platforms and message providers do not appear, because the core does not talk to them. The components match the modules in Section 4.3 and the schema rule in ADR-005.

7.6 AI issue 1: one large Fare and Ticketing component

Original prompt. * Using the case study, list the C4 Level 3 components for the Metrobus Core Platform with their responsibilities. (Reconstructed from memory; see Appendix A.2.)

AI recommendation. One component called Fare and Ticketing that handles fare calculation, ticket creation, QR tickets, travel cards, payment processing, offline validation and synchronization. A second component, Passenger Management, also manages travel-card information.

What was wrong. The component has seven responsibilities that change for different reasons. A new concession rule, a new payment provider and a change in device synchronization would all edit the same component. Travel-card data also has two owners, so it is unclear which component is correct when they disagree.

Knowledge used. Cohesion: a component should group things that change together. Data ownership: each piece of data needs exactly one owner. The case study expects frequent fare changes (passes, student fares, integrated fares), which makes the fare rules a separate change boundary.

Correction. I split it into Fare Engine, Card and Ticket Accounts, Validation Sync, Payment Adapter and Accounting Export. Card accounts are owned only by Card and Ticket Accounts. Passenger Accounts reads history through that component's interface.

7.7 AI issue 2: one shared container for all external integrations

Original prompt. * Suggest the C4 Level 2 containers for the Metrobus platform, including how it connects to payment, GPS, messaging and accounting systems. (Reconstructed from memory; see Appendix A.2.)

AI recommendation. A single Integration Adapters container that connects the platform to payment providers, banks, GPS providers, messaging providers, the accounting system, identity providers and partner systems.

What was wrong. A container is something that runs separately, so this box is a real runtime dependency. Payments, GPS, notifications and sign-in would all pass through it. If it failed, or if one slow provider used up its capacity, unrelated functions would fail together. This contradicts the case study rules that a tracking or notification failure must not affect ticketing. It also adds a deployable for a small team.

Knowledge used. Fault isolation: unrelated flows should not share a single point of failure. The C4 meaning of a container. The case study constraint that outside systems may be slow or unavailable.

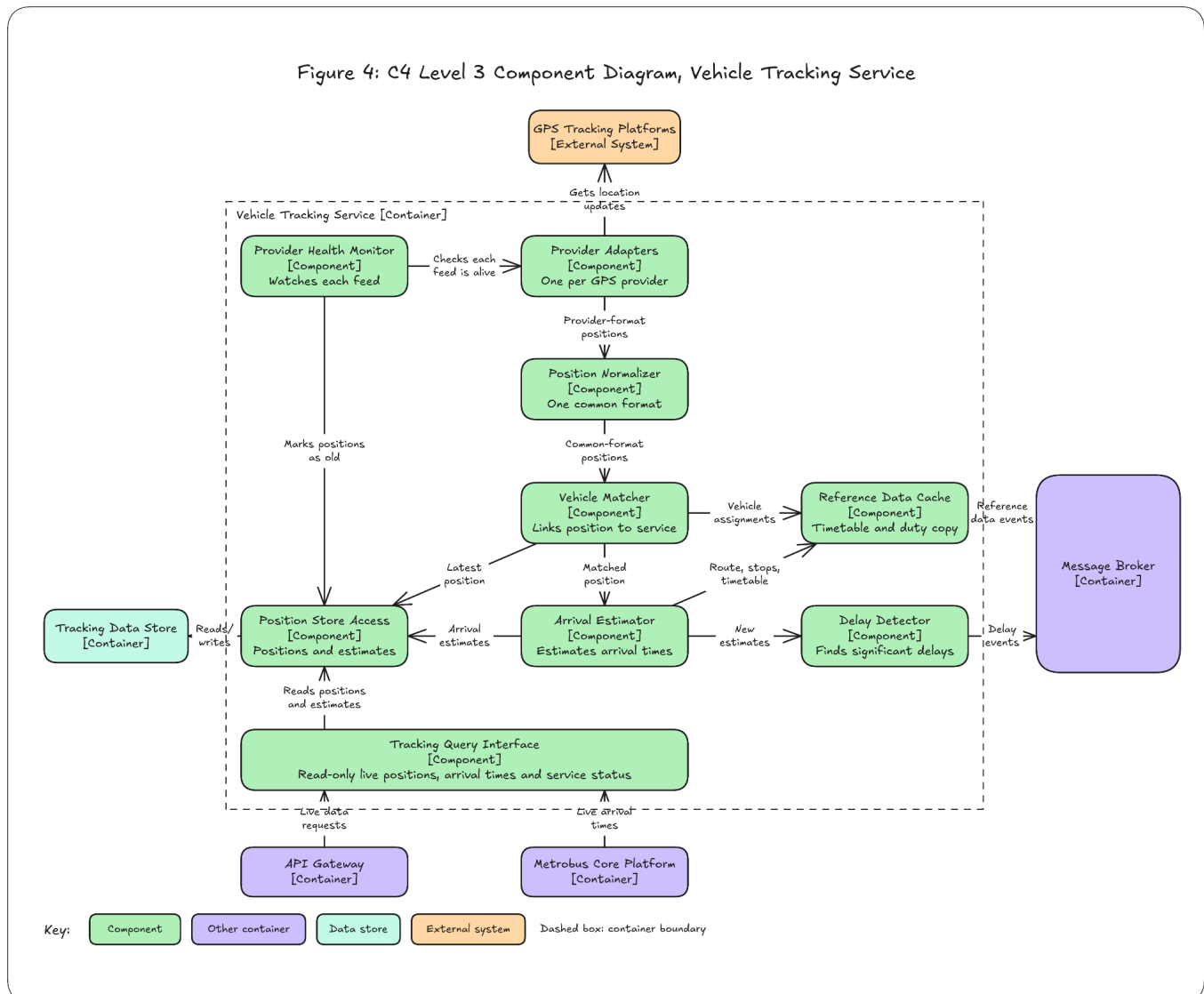
Correction. I rejected the shared container. Each adapter now lives inside the container that uses it (ADR-006). In the core these are Payment Adapter, Accounting Export and Partner Adapter.

8. Member 2: Vehicle Tracking Service

Responsible member: Ahamed M.N.K (IT24103829)

8.1 C4 Level 3 component diagram

The diagram opens the Vehicle Tracking Service container from Figure 2. Data flows in one direction: from the providers, through cleaning and matching, into the store, and out through one query interface. The nine components sit inside the container boundary. GPS Tracking Platforms, the Message Broker, the Tracking Data Store, the API Gateway and the Core Platform are shown outside it, matching the neighbours listed in Section 8.5.



Full-resolution diagram:

<https://github.com/chamals3n4/se3100-metrobus-architecture/blob/main/C4-Diagrams/Level-3-Components/SE40-C4-L3-Vehicle-Tracking-Service.png>

8.2 Component responsibilities and interactions

Components. The service has nine components.

Component	Responsibility
Provider Adapters	One adapter for each GPS provider. Each one handles that provider's API, sign-in and update rate.
Position Normalizer	Converts every provider format into one common position format. Rejects invalid, duplicate and out-of-order positions.
Vehicle Matcher	Links each position to a Metrobus vehicle and to the service that vehicle is running.
Position Store Access	Saves and reads the latest position and arrival estimates in the Tracking Data Store.
Arrival Estimator	Estimates arrival times at the next stops from the position, the route and the timetable.
Delay Detector	Compares estimates with the timetable. Publishes an event when a delay becomes significant and when it ends.
Reference Data Cache	Holds a copy of routes, stops, timetables, vehicle assignments and diversions. Updates it from core events.
Provider Health Monitor	Watches each provider's feed. Marks positions as old when updates stop and pauses calls to a failing provider.
Tracking Query Interface	Answers requests for live positions, arrival times and service status. Read-only.

Interactions. Positions flow in one direction. Provider Adapters get updates from the GPS Tracking Platforms and pass them to the Position Normalizer. The Vehicle Matcher links each cleaned position to a vehicle and its service, using the Reference Data Cache, and saves it through Position Store Access. The Arrival Estimator then calculates arrival times, saves them and passes them to the Delay Detector.

The service is linked to the rest of the platform in three ways. The Delay Detector publishes delay events to the Message Broker. The Reference Data Cache receives timetable, assignment and diversion events from the broker. The API Gateway and the Core Platform read live data through the Tracking Query Interface.

The Provider Health Monitor watches each adapter's feed and marks positions as old when updates stop. Position Store Access is the only component that uses the Tracking Data Store.

8.3 Design justification

Each design choice below is linked to the requirements, quality attributes or decisions that justify it.

Design choice	Justified by
One adapter per provider, all with the same output.	Providers differ in API, format and update rate, and more providers are expected. A new provider means one new adapter and no other change (TC2, FR19, QA9).
The Position Normalizer is separate from the adapters.	Provider differences end at one point. Everything after it works with one format (QA8).
The Vehicle Matcher links positions to services.	Passengers ask where their bus is, not where a GPS device is. Partner vehicles must appear the same as Metrobus vehicles (FR17, FR21).
The Reference Data Cache is fed by events.	The service does not call the core while processing positions, so it keeps working when the core is busy and adds no load to ticketing in the peak (QA2, ADR-002).
The Delay Detector publishes events.	The core and the Notification Service learn about delays without polling the tracking service (FR21, FR25, T4).
The Provider Health Monitor works per provider.	One provider failing must not affect vehicles tracked by another provider, and an old position must not be shown as live (QA2).
Only the latest position and short history are kept, in memory.	The case study asks for the latest available position. Many passengers read it in the peak, so reads must be fast (FR20, QA4).
The Tracking Query Interface is read-only and separate from the incoming path.	Heavy passenger reads cannot block incoming updates (QA4, OC3).

8.4 Design completeness

Requirement coverage.

Requirement	Component
FR2 (estimated arrival times)	Arrival Estimator, Tracking Query Interface
FR19	Provider Adapters, Position Normalizer
FR20	Vehicle Matcher, Position Store Access
FR21	Tracking Query Interface, Arrival Estimator, Delay Detector, Provider Health Monitor

Scenario 1: a normal location update.

1. A Provider Adapter gets a position from its provider.
2. The Position Normalizer converts it and checks that it is valid and newer than the last one.
3. The Vehicle Matcher finds the vehicle and its current service in the Reference Data Cache and saves the position.
4. The Arrival Estimator calculates arrival times for the next stops and saves them.
5. The Delay Detector sees the bus is now significantly late and publishes a delay event.
6. A passenger opens the app. The Tracking Query Interface returns the position and arrival time.

Scenario 2: one GPS provider stops responding.

1. The Provider Health Monitor sees no updates from that provider.
2. It marks the positions of that provider's vehicles as old.
3. The Tracking Query Interface returns those vehicles with the label "last known" and the client shows timetable times for them.
4. Vehicles from other providers are not affected.
5. The monitor retries the provider at intervals and removes the label when updates return.

Failure handling.

Failure	What the service does
A provider sends bad or out-of-order data	The Position Normalizer rejects it and keeps the last good position.
The core is unavailable	The service continues with its Reference Data Cache.
The Message Broker is unavailable	Positions and arrival times are still served. Delay events are retried.
The Tracking Data Store is unavailable	The Tracking Query Interface answers that live data is unavailable. Clients show timetable times.
The whole service is down	Journey Planning uses timetable times. Ticketing and validation are not affected.

8.5 Consistency with Levels 1 and 2

Each neighbouring element from the higher-level diagrams is checked against this design below.

Level 2 element	How it appears at Level 3
GPS Tracking Platforms	Reached only through Provider Adapters.
Tracking Data Store	Reached only through Position Store Access.
Message Broker	Events come in to the Reference Data Cache and go out from the Delay Detector.
API Gateway	Calls only the Tracking Query Interface.
Metrobus Core Platform	Calls only the Tracking Query Interface.

The service has no link to the Core Database, which matches the data ownership in Section 4.5. It has no link to ticketing, which is how the design meets the rule that tracking failure must not prevent ticket validation or fare processing.

8.6 AI issue 1: synchronous calls in both directions between tracking and the core

Original prompt. * Design the C4 Level 3 components for the Vehicle Tracking Service, including how it gets routes and timetables and how arrival times are estimated. (Reconstructed from memory; see Appendix A.2.)

AI recommendation. The ETA component fetches route and schedule data from the Core Platform with a request each time it needs it. The Core Platform also requests live arrival times from the tracking service. No fallback was described.

What was wrong. Each side needs the other to be running. If the core is slow, tracking cannot estimate arrival times. If tracking is down, journey planning in the core fails. The requests to the core also add load to the ticketing system during the peak. This contradicts the reason for separating tracking in the first place.

Knowledge used. Coupling: a synchronous call makes the caller only as available as the service it calls. A two-way dependency removes the benefit of separate deployment. Reference data changes rarely, so it can be copied.

Correction. I added the Reference Data Cache, which is updated by events from the core. The tracking service no longer calls the core. The one remaining call, from Journey Planning to the Tracking Query Interface, has a short timeout and a timetable fallback.

8.7 AI issue 2: no path for delay information to leave the tracking service

Original prompt. * Show how the containers communicate, including where the Message Broker is used. (Reconstructed from memory; see Appendix A.2.)

AI recommendation. The Message Broker connected only the Core Platform to the Notification Service. The tracking service had no way to publish anything.

What was wrong. The design was incomplete. The case study requires passengers to see significant delays and to be notified about them. Only the tracking service knows a bus is late. Without an outgoing path, the core would have to keep asking the tracking service, which adds coupling and finds delays late.

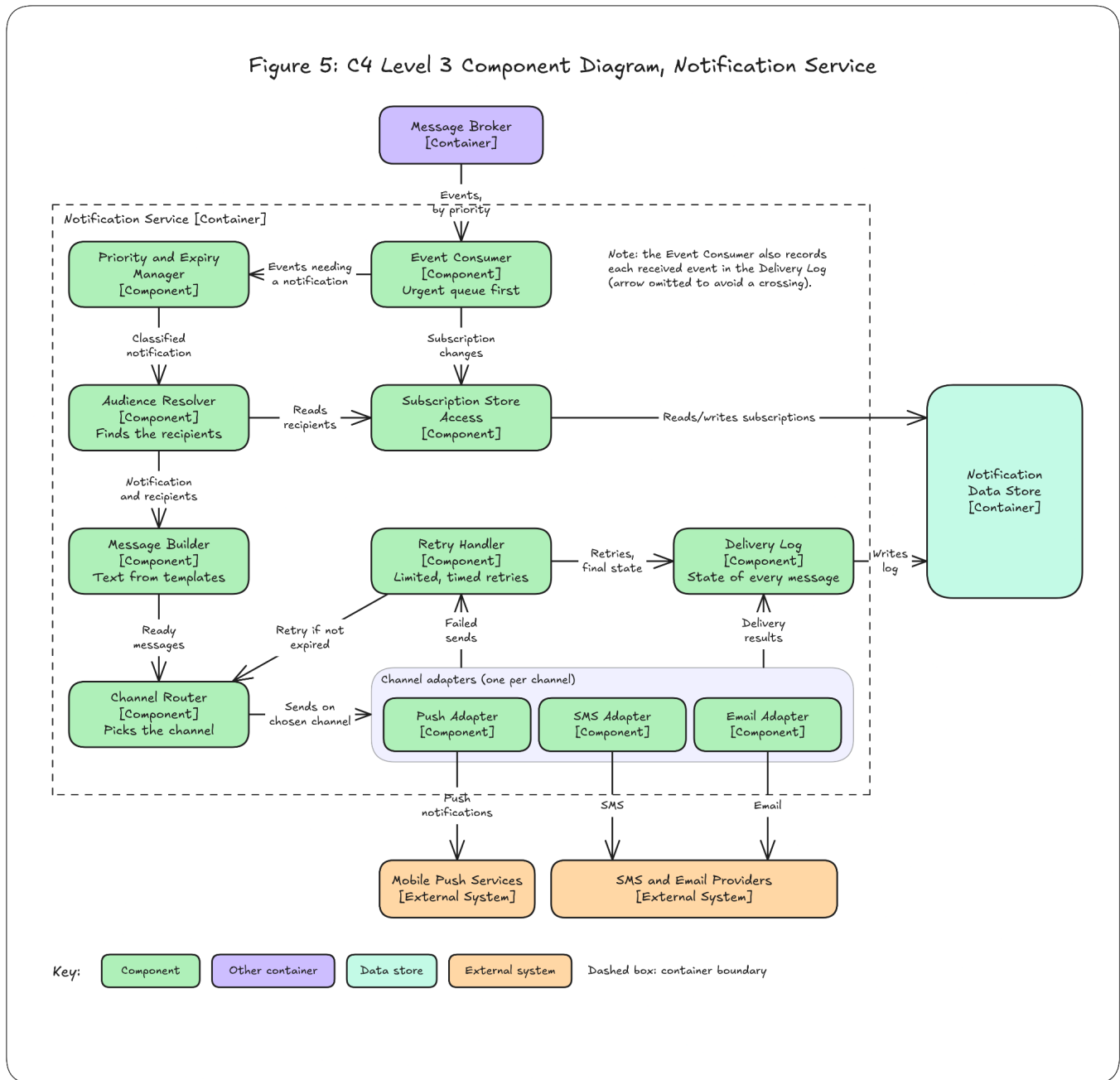
Knowledge used. In an event-driven design the component that owns a fact should publish it. Polling makes the caller depend on the service being available.

Correction. I added the Delay Detector, which publishes delay events to the Message Broker. The core and the Notification Service receive them. The Level 2 diagram was changed to show the tracking service publishing to the broker.

9. Member 3: Notification Service

Responsible member: Thilakarathna H.K.R.K (IT24103109)

9.1 C4 Level 3 component diagram



Full-resolution diagram:

<https://github.com/chamals3n4/se3100-metrobus-architecture/blob/main/C4-Diagrams/Level-3-Components/SE40-C4-L3-Notification-Service.png>

9.2 Component responsibilities and interactions

Components. The service has 11 components.

Component	Responsibility
Event Consumer	Reads events from the priority queues, always the urgent queue first. Ignores an event it has already handled.
Priority and Expiry Manager	Gives each notification a priority class and a time after which it is no longer useful.
Audience Resolver	Works out which passengers should receive the notification.
Subscription Store Access	Reads and updates the copy of passenger subscriptions, contact points and channel preferences.
Message Builder	Builds the message text for each channel from templates.
Channel Router	Chooses the channel from the passenger's preference and the priority. Chooses a second channel if the first fails.
Push Adapter	Sends push notifications through Mobile Push Services.
SMS Adapter	Sends SMS through the SMS provider.
Email Adapter	Sends email through the email provider.
Retry Handler	Retries failed messages with a growing wait. Stops at the expiry time or the attempt limit.
Delivery Log	Records the state of every message: sent, retried, failed or expired.

Interactions. A notification moves through the service in one line. The Event Consumer takes an event from the Message Broker and passes it to the Priority and Expiry Manager. The Audience Resolver finds the recipients through Subscription Store Access. The Message Builder prepares the text, and the Channel Router sends it to the Push, SMS or Email Adapter.

Each adapter calls its own provider. A failed send goes to the Retry Handler, which returns it to the Channel Router if it has not expired.

The Event Consumer also applies subscription changes from the core to Subscription Store Access. The Event Consumer, the adapters and the Retry Handler record results in the Delivery Log. Subscription Store Access and the Delivery Log are the only components that use the Notification Data Store.

9.3 Design justification

Each design choice below is linked to the requirements, quality attributes or decisions that justify it.

Design choice	Justified by
The Event Consumer reads the urgent queue first.	Time-sensitive disruption notifications must receive greater priority than informational or promotional messages (FR27).
Every notification has an expiry time.	A delay alert is wrong once the delay is over. An expired message is dropped, not sent late (FR26, QA4).
The Audience Resolver uses the service's own subscriptions copy.	The service does not call the core while sending, so a busy core cannot slow alerts and a busy notification service cannot slow the core (QA2, ADR-003).
One adapter for each channel, calling its provider directly.	A failing SMS provider does not affect push or email. A new provider or channel is one new adapter (TC1, QA9, ADR-006).
The Retry Handler has a growing wait and an attempt limit.	Provider failures are usually temporary, but endless retries would overload a recovering provider and deliver old news (TC1).
The Event Consumer ignores repeated events.	A broker can deliver an event twice. The passenger must receive one message.
The Channel Router can switch channel for urgent messages only.	An urgent alert should still arrive if push fails. SMS has a cost, so it is not used as a second channel for promotional messages.
The Delivery Log records every message.	Customer-support staff can check whether a passenger was told about a change or a card event (QA10).

9.4 Design completeness

Requirement coverage. FR26 is covered by the full path from Event Consumer to the three adapters. FR27 is covered by the priority queues, the Event Consumer and the Priority and Expiry Manager.

Every notification type in the case study has a priority class.

Priority class	Notification types from the case study
Urgent	Significant delays, route diversions, temporary stop closures, cancelled services, major disruptions
Normal	Changes affecting regular journeys, travel-card and payment events
Low	Informational and promotional messages

Scenario 1: a route diversion during a promotional campaign.

1. A promotional campaign has placed many events in the low queue.
2. An operations controller records a diversion. The core saves it and publishes a disruption event to the urgent queue.
3. The Event Consumer takes the urgent event before any remaining promotional event.
4. The Priority and Expiry Manager marks it urgent and sets its expiry to the expected end of the diversion.
5. The Audience Resolver finds passengers who follow the affected route.
6. The Channel Router sends push messages. For a passenger whose push fails, it sends an SMS.
7. The Delivery Log records each result. The promotional messages continue afterwards.

Scenario 2: the SMS provider is unavailable.

1. The SMS Adapter's request times out and it reports the failure to the Retry Handler.
2. The Retry Handler waits, then returns the message to the Channel Router.
3. After repeated failures the SMS Adapter stops calling the provider for a period.
4. Messages that reach their expiry time are dropped and recorded as expired.
5. Push and email messages are not affected. The operation in the core already completed before any of this started.

Failure handling.

Failure	What the service does
One provider is down	Only that channel is affected. Urgent messages switch channel. Others are retried until they expire.
All providers are down	Messages are retried until they expire. The core is not affected.
The Notification Data Store is down	Events stay in the Message Broker until the store returns.
The whole service is down	Events wait in the Message Broker. Operations in the core still complete.
The same event arrives twice	The Event Consumer ignores the second copy.

9.5 Consistency with Levels 1 and 2

Each neighbouring element from the higher-level diagrams is checked against this design below.

Level 2 element	How it appears at Level 3
Message Broker	The only way work enters the service, through the Event Consumer.
Notification Data Store	Reached through Subscription Store Access and Delivery Log.
Mobile Push Services	Reached only through the Push Adapter.
SMS and Email Providers	Reached only through the SMS Adapter and Email Adapter.

The service has no direct link to the Metrobus Core Platform, the Core Database or the API Gateway. This matches Figure 2 and ADR-003. It is also how the design meets the rule that a notification provider failure must not prevent the underlying operational change or ticketing transaction.

9.6 AI issue 1: no component handled priority

Original prompt. * Design the C4 Level 3 components for the Notification Service that sends push, SMS and email alerts to passengers. (Reconstructed from memory; see Appendix A.2.)

AI recommendation. A Notification Consumer passes events to a Notification Router that only decides the channel. All events arrive in one stream. The container description mentioned prioritization, but no component performed it.

What was wrong. With one stream, a large promotional send sits in front of an urgent disruption alert, and the alert waits. The case study requires the opposite. The design was also inconsistent: a responsibility named at Level 2 had no component at Level 3.

Knowledge used. In a single queue the first message in is the first out, so bulk traffic blocks urgent traffic. Each responsibility of a container at Level 2 must be traceable to a component at Level 3.

Correction. I used a separate queue for each priority, an Event Consumer that reads the urgent queue first, and a Priority and Expiry Manager that classifies each notification.

9.7 AI issue 2: retries with no limit and no expiry

Original prompt. * How should the Notification Service handle a failed SMS or push message? (Reconstructed from memory; see Appendix A.2.)

AI recommendation. A Retry and Failure Handler re-submits failed requests to the router. No wait between attempts, attempt limit or expiry time was described.

What was wrong. During a provider outage the failed messages pile up. When the provider returns, passengers receive a burst of old delay alerts that are no longer true, and the burst can overload the provider again.

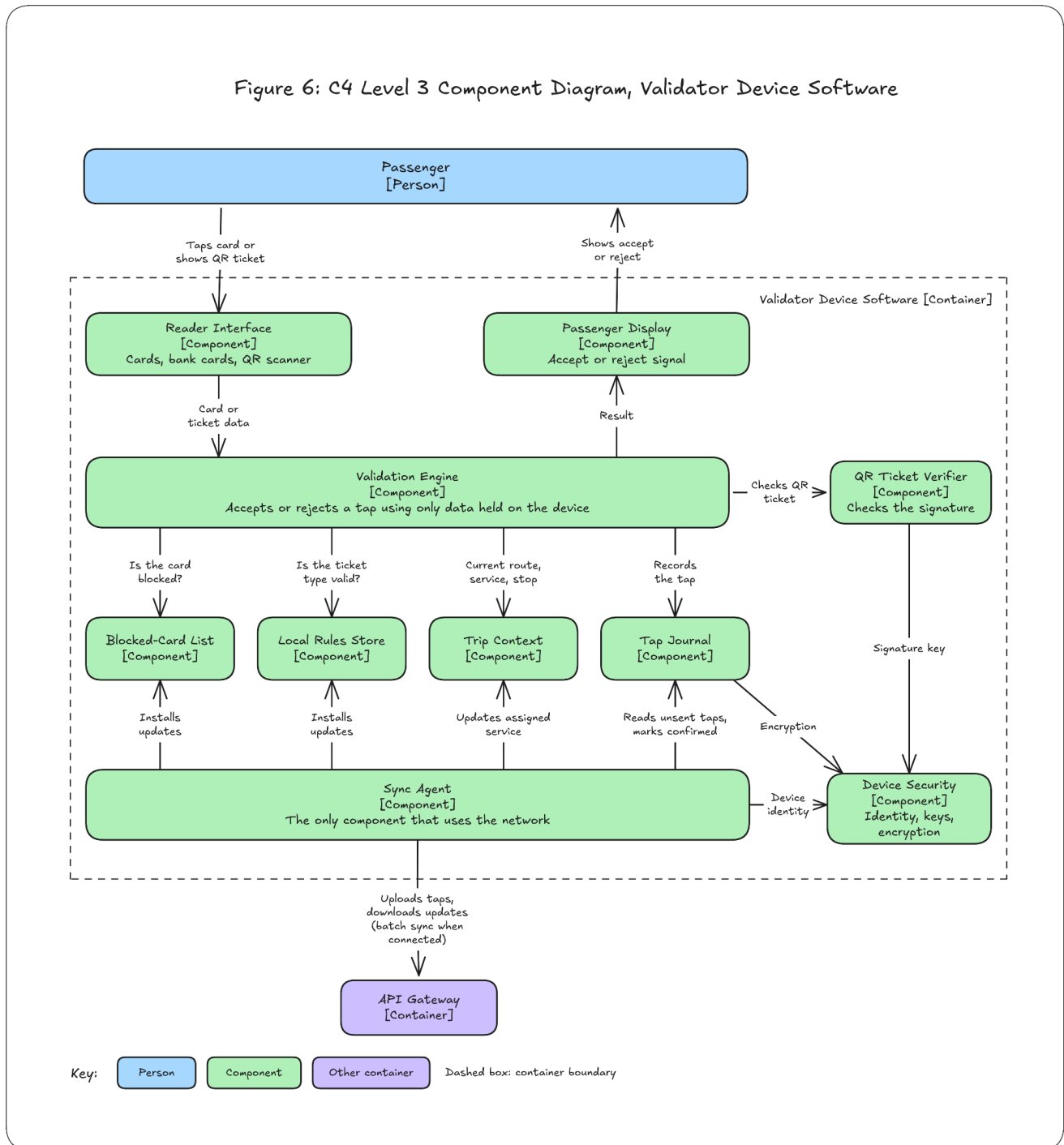
Knowledge used. A retry policy needs a growing wait, a maximum number of attempts and a time limit. Disruption information is time-sensitive, so late delivery can be worse than no delivery.

Correction. Every notification now has an expiry time. The Retry Handler waits longer after each attempt, stops at the expiry time or the attempt limit, and records the final state in the Delivery Log. Urgent messages switch channel before waiting.

10. Member 4: Validator Device Software

Responsible member: Dias M.B.N.S (IT24102244)

10.1 C4 Level 3 component diagram



Full-resolution diagram:

<https://github.com/chamals3n4/se3100-metrobus-architecture/blob/main/C4-Diagrams/Level-3-Components/SE40-C4-L3-Validator-Device-Software.png>

10.2 Component responsibilities and interactions

Components. The device software has ten components.

Component	Responsibility
Reader Interface	Reads contactless travel cards and bank cards, and scans QR tickets.
Validation Engine	Decides to accept or reject a tap, using only data held on the device.
Trip Context	Knows the current route, service and stop of this vehicle or gate.
Local Rules Store	Holds fare tables, the stop list and which ticket and pass types are valid, each with a version number.
Blocked-Card List	Holds the cards that must be rejected.
QR Ticket Verifier	Checks that a QR ticket carries a genuine Metrobus signature, is in date and was not already used on this device.
Tap Journal	Keeps an add-only record of every tap with a unique ID until the server confirms it.
Sync Agent	When connected, uploads taps, downloads new rules and lists, and reports device status. The only component that uses the network.
Device Security	Holds the device identity and keys, and encrypts the data stored on the device.
Passenger Display	Shows and sounds the accept or reject result.

Interactions. A tap is handled entirely on the device. The passenger taps a card or shows a QR ticket at the Reader Interface, which passes the data to the Validation Engine. The engine checks the Blocked-Card List, the Local Rules Store and, for QR tickets, the QR Ticket Verifier. It gets the current stop from Trip Context, records the tap in the Tap Journal and shows the result on the Passenger Display.

Synchronization runs separately. The Sync Agent uploads unsent taps from the Tap Journal through the API Gateway and marks the confirmed ones. It installs updates to the Local Rules Store and the Blocked-Card List, and updates the assigned service in Trip Context.

The Tap Journal, the Sync Agent and the QR Ticket Verifier use Device Security for encryption, the device identity and the signature key.

10.3 Design justification

Each design choice below is linked to the requirements, quality attributes or decisions that justify it.

Design choice	Justified by
The Validation Engine depends only on components on the device.	Validation must continue during loss of connectivity. Boarding speed is the same with or without a network (FR10, TC4, QA1, QA4).
The Sync Agent is the only component that uses the network.	A slow or lost connection cannot delay a tap. The network boundary is in one place (QA2).
The Tap Journal is add-only and gives every tap a unique ID.	No tap is lost if the device restarts, and the server can ignore a tap it receives twice (QA6).
The device records taps and the server calculates the final fare.	A transfer rule needs taps from other buses, which this device cannot know. Fare rules can change without replacing device software (FR8, FR9).
The Blocked-Card List is held on the device.	A lost or empty card can be rejected offline. The delay until the next update is the accepted cost in trade-off T5 (QA5).
QR tickets are checked by signature.	The device needs only a public key to detect a forged ticket, so no network is needed (FR7, QA5).
Device Security encrypts stored data, and bank cards are stored only as a token.	Validators are in public places and can be stolen. Payment data must be protected (QA5).

Trip Context is a separate component.	Fares depend on distance, so each tap must record the stop. We assume the device can read the stop from the driver's duty sign-on and the vehicle's position without the network (FR8, A1).
---------------------------------------	---

10.4 Design completeness

Requirement coverage.

Requirement or constraint	Component
FR7 (travel cards, bank cards, QR tickets at the door)	Reader Interface, Validation Engine, QR Ticket Verifier
FR10 (validate offline)	Validation Engine, Local Rules Store, Blocked-Card List, Tap Journal
FR10 (synchronize later)	Sync Agent, Tap Journal
BC3 (travel without the app)	Reader Interface
QA5 (protect payment data)	Device Security

Scenario 1: a tap while the bus has no network.

1. The passenger taps a travel card on the Reader Interface.
2. The Validation Engine checks the Blocked-Card List. The card is not listed.
3. It checks the Local Rules Store and gets the current stop from Trip Context.
4. It writes the tap to the Tap Journal with a unique ID.
5. The Passenger Display shows accept. The network was never used.

Scenario 2: the bus reconnects and the upload is interrupted.

1. The Sync Agent detects a connection and starts uploading unsent taps through the API Gateway.
2. The connection drops halfway.
3. The Sync Agent marks only the taps the server confirmed. The rest stay in the Tap Journal.
4. On the next connection it sends the remaining taps again. The server ignores any tap ID it already has.
5. The Sync Agent downloads a newer Blocked-Card List and installs it.

Failure handling.

Failure	What the device does
No network, or the whole back end is down	Works normally. Taps are stored.
Upload is interrupted	Unconfirmed taps stay in the Tap Journal and are sent again.
The device restarts	The Tap Journal is on permanent storage, so nothing is lost.
Tap Journal storage is filling up	The Sync Agent reports it and the device warns the driver well before it is full.
The Blocked-Card List is old	The device keeps validating and reports the list's age at the next synchronization, so staff can find devices that have not synchronized.
The device is stolen	Stored data is encrypted. The server stops accepting that device identity.

10.5 Consistency with Levels 1 and 2

Each neighbouring element from the higher-level diagrams is checked against this design below.

Higher-level element	How it appears at Level 3
Passenger (Level 1 and 2)	Interacts only with the Reader Interface and the Passenger Display.
API Gateway (Level 2)	Reached only by the Sync Agent, for batch synchronization.
Validation Sync in the core (Section 7)	The server-side partner of the Sync Agent. It removes duplicate taps and calculates fares.

The device has no direct link to the Core Database, the Vehicle Tracking Service or the Notification Service, which matches Figure 2. The design follows ADR-004 and is the main way the architecture meets its first-ranked quality attribute, availability of ticket validation.

10.6 AI issue 1: offline validation placed in a server component

Original prompt. * Suggest the containers and components for ticketing, including offline validation on buses and synchronization later. (Reconstructed from memory; see Appendix A.2.)

AI recommendation. Offline validation and synchronization were listed as responsibilities of the Fare and Ticketing component inside the Core Platform. The validator appeared as a box in the container diagram but was missing from the container list and descriptions.

What was wrong. The Core Platform runs on a server. When the network is down, the validator cannot reach it, so a server component cannot perform offline validation. The requirement was named but not met. The device was treated as a simple client with no logic or storage of its own.

Knowledge used. At Level 2 a container shows where code runs. A function is only as available as every part it depends on. Validation that depends on the device, the mobile network, the gateway and the core fails when any one of them fails.

Correction. I made the Validator Device Software a container of its own, with the decision logic and storage on the device. The core keeps only the server side of synchronization (Validation Sync).

10.7 AI issue 2: permission checks placed in the API Gateway

Original prompt. * Design the components of the API Gateway for the Metrobus platform, including authentication and authorization. (Reconstructed from memory; see Appendix A.2.)

AI recommendation. The API Gateway contains an Authorization component that checks whether the user has permission to access the requested resource. The same design also said the gateway should hold no business logic.

What was wrong. Metrobus permissions depend on business data. Partner staff should manage only their own company's vehicles. A fleet administrator should not issue refunds. To decide this, the gateway would need to know vehicles, companies and roles, which is business logic. The recommendation contradicted itself, and the rules would end up in two places.

Knowledge used. Separation of concerns: a gateway handles checks that are the same for every request, such as whether the token is valid. A decision that depends on domain data belongs with the module that owns that data. The case study requires each staff category to see only what its responsibilities need.

Correction. The API Gateway only checks the sign-in token and limits request rates. Roles and permissions are checked in the core by Access and Administration, beside the data they protect (ADR-007). Validators are identified as devices, not as users.

11. Conclusion

The proposed architecture is a domain-partitioned modular monolith with two separate services and offline-first validators. It was chosen because the case study asks for two things that pull in opposite directions: strong isolation of failures, and a platform simple enough for a small team to run.

Each separately deployed part answers a specific case study statement. The Vehicle Tracking Service exists because tracking failure must not stop ticketing. The Notification Service exists because a provider failure must not block an operation. The validators decide locally because validation must continue without the network. Everything else stays in one core, because nothing in the case study justifies separating it.

The design accepts three costs and states them openly. A card with no balance may be accepted while a device is offline. Reports may be slightly behind live data. A serious fault in the core affects all core modules, which is reduced by running more than one copy and by the offline validators.

The C4 Level 1, 2 and 3 views describe the same system at different levels of detail. Each Level 3 diagram opens one container from Level 2 and uses only the neighbours shown there. Future growth to other cities, new providers and new fare products is supported by the network attribute, the adapters and the separate Fare Engine.

The design follows the ranked quality attributes. Offline validators and two copies of the core protect ticket validation, one Fare Engine and unique tap IDs protect fare integrity, and separate adapters keep the platform easy to change.

Reviewing the AI recommendations critically also improved the design. Each of the eight issues was corrected by checking it against the case study and principles such as cohesion, fault isolation and clear data ownership.

As next steps, the assumptions in Section 1.2 should be confirmed with Metrobus, and offline validation should be tested as a prototype. Because each core module has its own schema, a module can later become a separate service without redesigning the rest of the platform.

References

1. SE3100 Architecture Based Development, Case Study-Based Group Assignment specification, SLIIT, 2026.
2. SE3100 Architecture Based Development, Case Study 01: Sri Lanka Metrobus Service Platform, SLIIT, 2026.
3. SE3100 Architecture Based Development, lecture materials, SLIIT, 2026.
4. S. Brown, The C4 model for visualising software architecture. Available: <https://c4model.com>
5. S. Brown, The C4 Model: Visualizing Software Architecture, O'Reilly Media, 2026.
6. S. Brown, Software Architecture for Developers, Volume 2: Visualise, document and explore your software architecture, Leanpub.
7. S. Brown, Structurizr documentation. Available: <https://docs.structurizr.com>
8. Markdown Architectural Decision Records (MADR). Available: <https://adr.github.io/madr>
9. M. Nygard, "Documenting Architecture Decisions," Cognitect blog, 15 November 2011.
10. M. Richards and N. Ford, Fundamentals of Software Architecture, O'Reilly Media, 2020.
11. L. Bass, P. Clements and R. Kazman, Software Architecture in Practice, 4th edition, Addison-Wesley, 2021.
12. S. Newman, Building Microservices, 2nd edition, O'Reilly Media, 2021.
13. C. Richardson, Microservices Patterns, Manning Publications, 2018. Pattern catalogue (API Gateway, Transactional Outbox, Circuit Breaker) available: <https://microservices.io/patterns>
14. M. Fowler, "MonolithFirst," martinowler.com, 2015. Available: <https://martinfowler.com/bliki/MonolithFirst.html>
15. M. Fowler, "CircuitBreaker," martinowler.com, 2014. Available: <https://martinfowler.com/bliki/CircuitBreaker.html>
16. M. Kleppmann, Designing Data-Intensive Applications, O'Reilly Media, 2017.
17. E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software, Addison-Wesley, 2003.
18. ISO/IEC 25010:2023, Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE), Product quality model, ISO, 2023.

Appendix A: AI prompts used

The assignment requires every AI prompt to be disclosed. A.1 lists the prompts from the review session that produced this version of the report. A.2 is for the prompts each member used earlier.

A.1 Group review session

Tool: Claude (Anthropic). Date: 6 and 7 October 2026. The assignment specification, the case study and the first draft of the report were attached. Prompts are copied exactly as typed.

No.	Prompt	Used for
1	so here is the assignment for the ABD module and now we got this case study number 1 and one of our team members start writing the docs most of the things so before the submission as the group leader i need to verify these works and can you read the assignment specification and the case study we got can you give me is this correct and the architecture patterns and things before going to anything just tell me is the correct way her wrote	Review of the first draft
2	before telling me what this assignment specification tells and what is the case study we got and what we need to do ? first tell this	Understanding the assignment
3	then think as the softwre archtie and tell me is this my teammmember wrote doc is correct i mean the architecut and thing ? what is he select and tell me only that part	Review of the selected style
4	no i mean is this case study tell this archtie to use or we got selected i didnt read it thats why aske	Understanding the assignment
5	forgot about the doc we wrote so if you the software archite tell me which one is to use	Style selection
6	well now clearly explain the case study we got assigned more clearly a-z	Understanding the case study
7	well now what is the assignment specification document say us to do explain it also clearly NOW	Understanding the assignment

8	can you explain what is c4 mean?	Understanding the C4 model
9	well now we need to update this document neh ? so now accordin tothe assignmen specifianadn thing what is the TOC should be to be great makr i mean best in best	Report structure
10	well now can you wreit hr full updated diagram please.i mean even with the full content but in clear simple english now you know the thing and how to do te thn and now write but keep the diagram i will add them later	Drafting the report text
11	now od the other remaming 3 members everything also	Drafting Sections 8 to 10
12	now add the 11. Conclusion References(add some useful things from th internet like simsons docs and thin dont add the brakel one like [KEEP only the sources the team actually used, and add the module's lecture materials.] also Appendix A: AI prompts used	Conclusion, references and Appendix A
13	take these things to new page and make things clear professo please thesething bit messy neh	Page layout
14	try t add those things into since page and also add some cler sapce bet adn t table cells i mean bcz now feel like the entire doc with table feeling right so if can add some small lines of content some place as wel you think better	Table spacing and short lead-in text
15	well now i need a TOC in the beginning right	Table of contents
16	but now this is too much bit reduce and add space	Shortening the conclusion

A.2 Prompts used by each member

Each member lists the AI prompts they used while preparing their part of the report.

Member	Tool	Prompt	Used for
Senarathna Y.M.C.S	Claude (Anthropic)	* Using the case study, list the C4 Level 3 components for the Metrobus Core Platform with their responsibilities.	First component design for the Metrobus Core Platform (Section 7.6)
Senarathna Y.M.C.S	Claude (Anthropic)	* Suggest the C4 Level 2 containers for the Metrobus platform, including how it connects to payment, GPS, messaging and accounting systems.	First container design (Section 7.7)
Senarathna Y.M.C.S	Claude (Anthropic)	* Which functional requirements does each Core Platform component cover? Make a table.	Requirement coverage for the Core Platform (Section 7.4)
Senarathna Y.M.C.S	Claude (Anthropic)	* What happens in the Core Platform if the payment provider or the accounting system is down?	Failure handling for the Core Platform (Section 7.4)
Ahamed M.N.K	Gemini (Google)	* Design the C4 Level 3 components for the Vehicle Tracking Service, including how it gets routes and timetables and how arrival times are estimated.	First component design for the Vehicle Tracking Service (Section 8.6)
Ahamed M.N.K	Gemini (Google)	* Show how the containers communicate, including where the Message Broker is used.	First container design and the Message Broker links (Section 8.7)

Ahamed M.N.K	Gemini (Google)	* How can the tracking service support several GPS providers with different data formats?	Provider Adapters and Position Normalizer (Section 8.2)
Ahamed M.N.K	Gemini (Google)	* What should the tracking service show passengers when a GPS provider stops sending updates?	Failure handling for the Vehicle Tracking Service (Section 8.4)
Thilakarathna H.K.R.K	Gemini (Google)	* Design the C4 Level 3 components for the Notification Service that sends push, SMS and email alerts to passengers.	First component design for the Notification Service (Section 9.6)
Thilakarathna H.K.R.K	Gemini (Google)	* How should the Notification Service handle a failed SMS or push message?	Retry and failure handling design (Section 9.7)
Thilakarathna H.K.R.K	Gemini (Google)	* How can urgent disruption alerts be sent before promotional messages?	Priority handling in the Notification Service (Section 9.3)
Thilakarathna H.K.R.K	Gemini (Google)	* How do we stop a passenger getting the same notification twice?	Duplicate event handling (Section 9.3)
Dias M.B.N.S	ChatGPT (OpenAI)	* Suggest the containers and components for ticketing, including offline validation on buses and synchronization later.	First container and component design for ticketing (Section 10.6)
Dias M.B.N.S	ChatGPT (OpenAI)	* Design the components of the API Gateway for the	API Gateway design

		Metrobus platform, including authentication and authorization.a	(Section 10.7)
Dias M.B.N.S	ChatGPT (OpenAI)	* What data does the validator need to store so it can accept or reject a card without the network?	Local Rules Store and Blocked-Card List (Section 10.2)
Dias M.B.N.S	ChatGPT (OpenAI)	* How can QR tickets be checked on the device without contacting the server?	QR Ticket Verifier (Section 10.3)